1. Overview of the Existing Chatbot Implementation

Chatbot functionality is provided through the existing Your Accessible Flight (YAF) backend using the API endpoint POST /api/ai/chat.

The message sent by the user to the chatbot is sent in JSON form as a request which is then processed by the AI service from the backend.

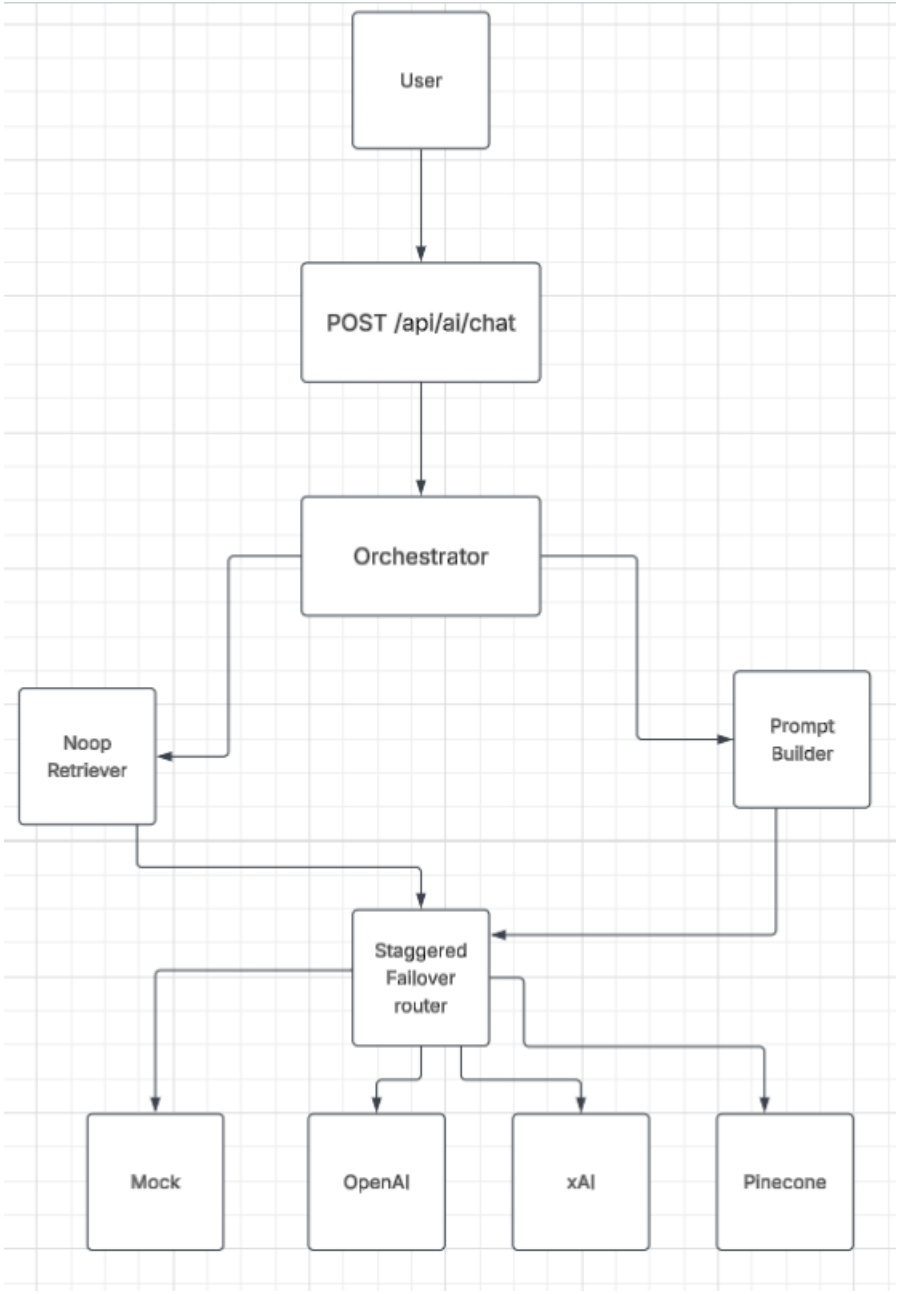
The AI handler gets the request initially and authenticates it. Then an AI orchestrator gets the request. The prompt needs to be prepared by the orchestrator and sent to the chosen AI provider using a staggered failover router.

The four AI providers that are currently integrated with the backend include Pinecone, OpenAI, xAI, and Mock. In the local development configuration, where AI_PROVIDERS=mock, the Mock provider is chosen. As such, the testing done locally helps confirm that the API for the chatbot is working fine, but this time round the output is a dummy one and not from an actual AI model.

Consequently, the present chatbot has the basic provider and API framework to support an AI assistant, although it lacks the retrieval and grounding capabilities to connect the chatbot with the application data.

2. Existing Chatbot Structure

The current chatbot architecture can be represented as follows:



The AI Handler's role is to accept the HTTP request, perform authentication, validate the input message, and forward the request to the orchestrator.

Orchestrator tries to retrieve data before creating the actual prompt, and then passes the prompt to the staggered failover router. The staggered failover router takes care of all the AI Providers and provides failover capability in case of failure of an AI Provider.

Even though there is a retrieval interface within the AI system, the current configuration of the application employs NoopRetriever. Thus, no contextual data is being fetched prior to generating an AI response.

3. AI API and Database Connections

3.1 AI API

Chatbot access is provided:

POST /api/ai/chat

A typical request contains:

```
{
  "content": "What accessibility services does the Sydney airport have?"
}
```

This endpoint is provided in the authenticated /api endpoints. For local development, authentication is bypassed by setting:

AUTH_ENABLED=false

The backend system therefore uses its development mode of authentication.

The AI provider is set using the environment variable AI_PROVIDERS. The list of available providers is:

- Mock
- OpenAI
- xAI
- Pinecone

3.2 database

PostgreSQL is the backend database used. The data stored in the database is structured and is highly important for the chatbot, and includes:

- Airline information
- Airline accessibility information
- Airline equipment guidelines
- Battery restrictions
- Airport information
- Airport accessibility services
- Equipment types
- User's equipment information

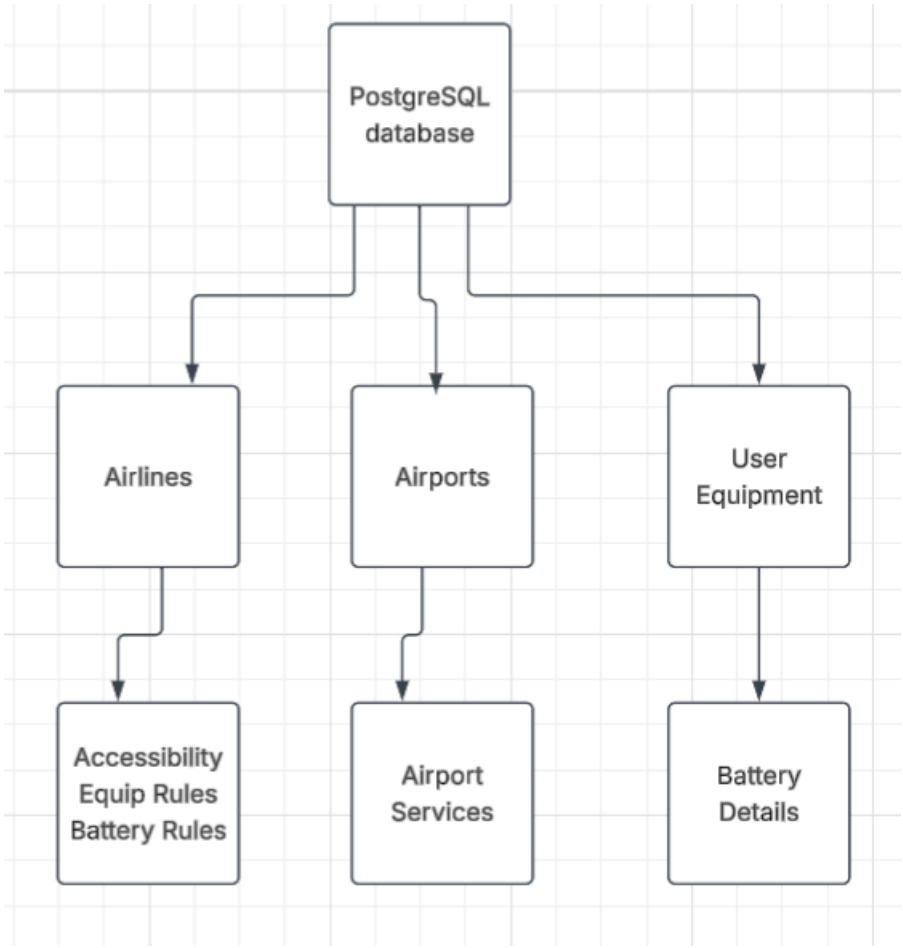
For instance, the data stored in the database includes wheelchair measurements, equipment weight restrictions, batteries types, battery capacities and restriction on the number of batteries.

The user_equipment table also holds data that can be used for personalized accessibility assistance.

3.3 current AI-database connection

While there is accessible information stored within the PostgreSQL database, the current chatbot does not retrieve such information directly.

The current database schema is:



The current AI architecture is the first diagram from **Section 2**

There are currently no connections between the PostgreSQL accessibility data and the retrieving process of the chatbot.

The repository has a Pinecone retriever whose retrieving features are yet to be implemented. Thus, the chatbot is not currently making use of the database's accessibility data through the retrieving process.

4. Relevant Areas for Sprint 2

4.1 Connecting the Chatbot to Accessibility Data

The existing data in the PostgreSQL database includes structured and possibly authoritative information regarding accessibility. An important point in Sprint 2 is the way to get this information to the chatbot.

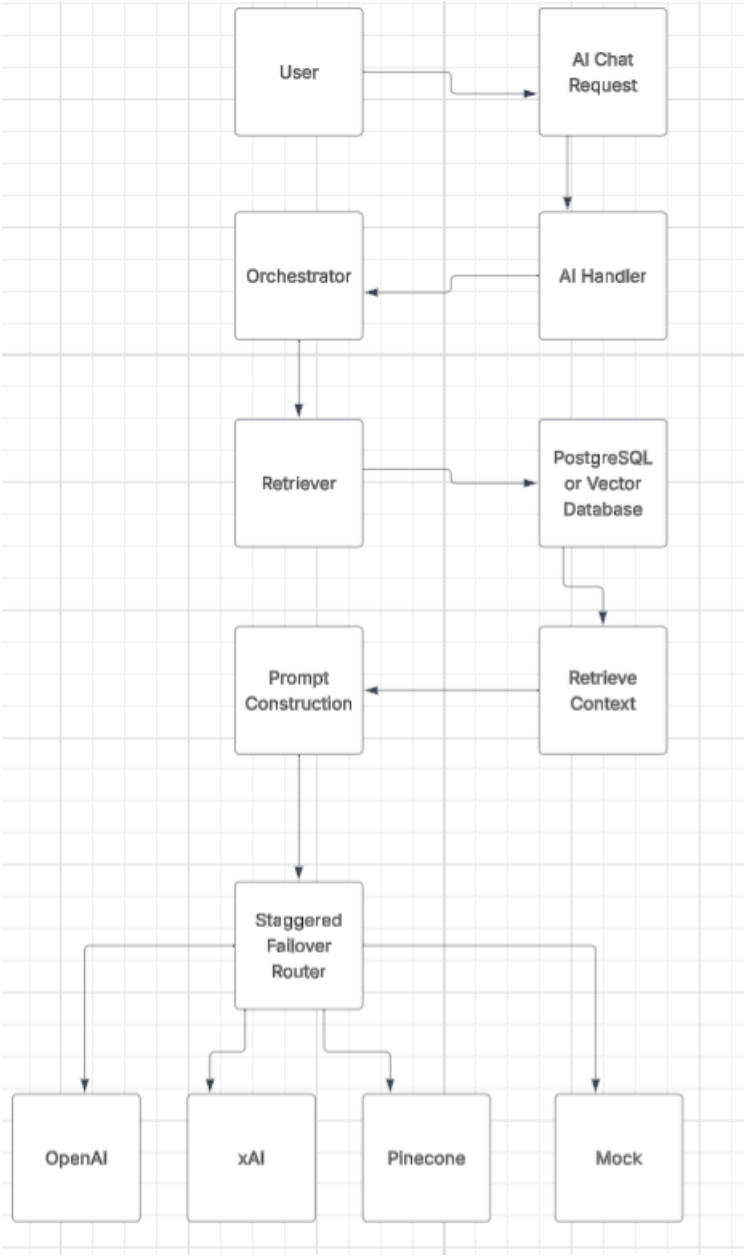
Using the chatbot to get access to such data will decrease the chances that the chatbot will use its general knowledge only.

4.2 Implementing Retrieval and Grounding

Current NoopRetriever indicates that the bot doesn't retrieve information before giving a reply.

There is a Pinecone retriever in the repo but it currently gives ErrNotImplemented. Sprint 2 needs to look into this and implement the proper retrieval strategy.

The intended high-level flow could be:



Implementation of exact retrieval is to be decided in Sprint 2 depending on the requirements and limitations of the project.

4.3 Personalised Accessibility Information

The database has the equipment details for individual users such as equipment dimensions, weight and battery details.

This provides the ability to offer more personalized responses by the chatbot. For instance, the application can utilize the equipment details of the user to determine if certain airline restrictions apply.

It is essential to make sure that any implementation provides the capability for users to only access their personal details.

4.4 Accessibility Rule Checking

The database includes rules related to airline equipment and battery dimensions and weight, as well as the battery capacity and number of batteries.

These rules are deterministic and should be treated independently of AI-generated responses.

A potential approach would be:

It would enable deterministic rules to determine if the item satisfies the conditions, while giving the AI an opportunity to explain the result naturally.

4.5 Improving Chatbot Response Accuracy

The present prompt already indicates that the chatbot is an accessibility-oriented flight assistant and should not guess if there is no answer in the given context.

Sprint 2 needs to be centered around providing relevant and precise context to the prompt. It will make the current instruction more efficient and minimize the possibility of delivering unsupportable accessibility information to the users.

5. Key Technical Considerations

Accuracy of Data

Accessible data has to be accurate since inaccurate data might affect the accessibility needs of the travellers. The chatbot should thus use data that is validated by applications where necessary.

Personalization

The current user_equipment table gives room for personalized chatbot replies. Personal information should however be provided to the chatbot when it is needed and authorized by the user.

AI Provider Reliability

The current staggered failover router provides a solution for provider failure. Any Sprint 2 enhancement should keep this reliability and also consider retrieval service failure.

Authentication and Privacy

Chatbot gets the identifier of the user from the authentication service. In case of use of equipment data specific to the user in the chatbot responses, suitable authentication and authorization controls would have to be in place.

Current AUTH_ENABLED=false setting is only for local development and should not be taken as a production security configuration.

Local and Production Environment

Current local configuration includes the following: AUTH_ENABLED=false, AI_PROVIDERS=mock

This way it is possible to test the existing chatbot API without any need of credentials of external AI services. Nevertheless, using Mock provider does not show how the AI provider really behaves.

6. Summary

The current implementation of the YAF backend gives a functional basis for building an accessible chatbot. The chatbot is made available via the /api/ai/chat API endpoint and works by processing requests via an AI handler, orchestrator and staggered failover router. The current implementation allows multiple AI providers, specifically Mock, OpenAI, xAI and Pinecone.

The PostgreSQL database holds a wide variety of accessibility data on airlines, airports, mobility equipment and battery restrictions, as well as user-specific equipment data. Currently, however, none of that data is connected to the chatbot retrieval procedure. The current implementation uses NoopRetriever, whereas Pinecone Retriever is available but has not been implemented yet.

Therefore, the main concerns of Sprint 2 relate to providing a reliable retrieval/grounding mechanism, connecting the chatbot to verified accessibility data, exploring personalized responses based on the user equipment data, separating the deterministic accessibility rules from the AI-generated explanations, and providing authentication and privacy guarantees.